DSA Assignment 1 Report

Name- Dhiren Yadav

Roll No.- 2501730271

Course- B.Tech CSE (AI/ML)

Section- A

Submitted To- Ms. Neetu Chauhan

Part A: Stack ADT (ADT + Operations)

What is an ADT?

An Abstract Data Type (ADT) is a logical model that defines what operations can be performed on a data structure without describing how they are implemented. It focuses on the behavior of the data structure rather than its internal implementation. For example, a Stack ADT defines operations like push, pop, and peek, but it does not specify how the stack is stored in memory. This allows different implementations while keeping the same functionality.

Why are stack operations usually O(1)?

Stack operations such as push and pop are usually O(1) because they only involve adding or removing an element from the top of the stack. These operations do not require searching or shifting other elements. Since the time taken does not depend on the number of elements in the stack, the complexity remains constant.

Part B: Factorial & Fibonacci (Recursion + Complexity)

1. Time Complexity (Big-O) and Space Complexity (Big-O)

For factorial, the time complexity is O(n) because the function calls itself once for every value until it reaches 0. The space complexity is also O(n) because each recursive call is stored in the call stack.

For naive Fibonacci, the time complexity is $O(2^n)$ because each call creates two more calls, so the number of calls grows very quickly. The space complexity is O(n) because the recursion depth can go up to n.

2. Big-Ω and Big-Θ

For factorial, the best, average, and worst cases are the same, so Big-Ω = Θ(n).

For naive Fibonacci, the number of recursive calls grows exponentially, so Big-Ω ≈ $\Theta(2^n)$. This means the algorithm takes exponential time in most cases.

3. Why naive Fibonacci becomes slow for large n

Naive Fibonacci becomes slow because it calculates the same Fibonacci numbers many times. For example, when computing fib(5), the function repeatedly recalculates fib(3) and fib(2). As n increases, these repeated calculations grow very quickly, which makes the algorithm very slow.

Part C: Tower of Hanoi (Recursion + Trace)

1. Why number of moves becomes 2n−12

In Tower of Hanoi, to move n disks, we first move n−1 disks to the auxiliary rod, then move the largest disk, and finally move the n−1 disks again to the destination rod. Because the problem repeats twice for n−1 disks, the total moves become $2^n - 1$.

2. Time Complexity

The time complexity is $O(2^n)$ because the number of recursive calls doubles for each additional disk. As the number of disks increases, the number of moves grows exponentially.

3. Space Complexity

The space complexity is O(n) because recursion stores function calls in the call stack. The maximum recursion depth is equal to the number of disks n.

Part D: Recursive Binary Search (Divide & Conquer + Recurrence)

1. Time Complexity Using Recurrence

In binary search, the list is divided into two halves each time. The algorithm only searches in one half of the list. The recurrence relation is:

$T(n) = T(n/2) + O(1)$

Solving this gives the time complexity O(log n).

2. Best Case

The best case is O(1). This happens when the element we are searching for is exactly the middle element of the array.

3. Average Case

The average case is O(log n) because the search space keeps reducing by half each time.

4. Worst Case

The worst case is O(log n) when the element is at the last level of the search or not present in the list.

Part E: Short Paradigm Reflection

1. Binary Search – Divide & Conquer (Why?)

Binary search follows the Divide and Conquer paradigm because the algorithm repeatedly divides the sorted array into two halves. It checks the middle element and decides whether the search should continue in the left half or the right half of the array. This process keeps reducing the problem size by half until the element is found or the search space becomes empty. Because the input size keeps halving at every step, the overall time complexity becomes O(log n).

2. Fibonacci Memoized – Dynamic Programming

The memoized Fibonacci algorithm demonstrates the concept of Dynamic Programming. In the naive recursive Fibonacci method, the same Fibonacci values are calculated many times, which creates overlapping subproblems and makes the algorithm inefficient. Memoization solves this issue by storing the results of previously computed Fibonacci numbers in memory. When the same value is required again, the stored result is used instead of recomputing it. This avoids repeated work and improves the efficiency of the algorithm from $O(2^n)$ to O(n).

3. Tower of Hanoi – Recursion

Tower of Hanoi is a classic example of recursion where a problem is solved by breaking it into smaller instances of the same problem. The base case occurs when there is only one disk, which can be moved directly from the source rod to the destination rod. For multiple disks, the algorithm first moves the top n−1n-1n−1 disks from the source rod to an auxiliary rod, then moves the largest disk to the destination rod, and finally moves the n−1n-1n−1 disks from the auxiliary rod to the destination rod. In this way, the large problem is solved by repeatedly solving smaller subproblems.

4. Real-Life Examples

Divide & Conquer

•Searching for a word in a dictionary by opening the middle and deciding which half to search.

Dynamic Programming

•Route planning in navigation apps, where the best paths between locations are stored and reused.

Greedy Approach

•Giving change in a store, where the largest currency denomination is given first, followed by smaller denominations until the amount is completed.